vLLM 병렬 처리 및 분산 추론 최적화

개요

vLLM은 대규모 언어 모델의 효율적인 추론을 위해 다양한 병렬 처리 기법을 활용합니다. 단일 GPU로는 처리하기 어려운 대형 모델을 여러 GPU에 분산하여 처리하거나, 처리량을 향상시키기 위해 여러 GPU를 병렬로 활용하는 전략을 제공합니다. 본 문서에서는 vLLM의 핵심 병렬 처리 기법인 텐서 병렬화(Tensor Parallelism)와 파이프라인 병렬화(Pipeline Parallelism)의 구현 방식과 성능 최적화 전략을 상세히 분석합니다.

병렬 처리의 기본 개념

텐서 병렬화 (Tensor Parallelism)

텐서 병렬화는 모델의 각 레이어를 여러 GPU에 분할하여 처리하는 방식입니다. 특히 Transformer 모델의 어텐션 메커니즘과 피드포워드 네트워크에서 효과적으로 활용됩니다.

핵심 원리:

- 행렬 연산을 여러 GPU에 분산하여 병렬 처리
- 각 GPU가 전체 텐서의 일부분을 담당
- GPU 간 통신을 통해 결과를 집계

파이프라인 병렬화 (Pipeline Parallelism)

파이프라인 병렬화는 모델의 레이어를 여러 GPU에 순차적으로 분배하여 처리하는 방식입니다. 각 GPU가 서로 다른 레이어를 담당하며, 데이터가 파이프라인을 통해 순차적으로 처리됩니다.

핵심 원리:

- 모델 레이어를 GPU별로 분할

- 순차적 데이터 처리를 통한 메모리 효율성 향상
- 파이프라인 버블 최소화를 통한 처리량 최적화

ParallelConfig를 통한 병렬 처리 설정

vLLM의 병렬 처리는 `ParallelConfig` 클래스를 통해 설정됩니다. 이 클래스는 다양한 병렬화 옵션을 제공합니다.

```
@dataclass
class ParallelConfig:
"""Configuration for the distributed execution."""
pipeline_parallel_size: int = 1
"""Number of pipeline parallel groups."""
tensor_parallel_size: int = 1
"""Number of tensor parallel groups."""
data_parallel_size: int = 1
"""Number of data parallel groups."""
distributed_executor_backend: Optional[Union[DistributedExecutorBackend,
type["ExecutorBase"]]] = None
"""Backend to use for distributed model workers."""
world_size: int = field(init=False)
"""world_size is TPxPP, it affects the number of workers we create."""
```

병렬 처리 설정 계산

```
@property
def world_size_across_dp(self) -> int:
"""world_size_across_dp is TPxPPxDP, it is the size of the world
including data parallelism."""
return self.world_size * self.data_parallel_size
```

전체 워커 수는 텐서 병렬 크기와 파이프라인 병렬 크기의 곱으로 결정되며, 데이터 병렬까지 고려하면 더 많은 워커가 필요합니다.

ExecutorBase와 분산 실행 관리

ExecutorBase 아키텍처

`ExecutorBase` 는 vLLM의 모든 실행자(executor)의 기본 클래스입니다. 단일 디바이스 또는 다중 디바이스에서 모델을 실행하는 책임을 담당합니다.

```
class ExecutorBase(ABC):
"""Base class for all executors.

An executor is responsible for executing the model on one device,
or it can be a distributed executor
that can execute the model on multiple devices.
"""

uses_ray: bool # whether the executor uses Ray for orchestration.
supports_pp: bool = False # whether the executor supports PP
```

```
def __init__(self, vllm_config: VllmConfig) -> None:
self.vllm_config = vllm_config
self.model_config = vllm_config.model_config
self.cache_config = vllm_config.cache_config
self.parallel_config = vllm_config.parallel_config
# ... 기타 설정
```

분산 RPC 통신

ExecutorBase는 `collective_rpc` 메서드를 통해 모든 워커에서 동일한 작업을 실행합니다:

```
@abstractmethod
def collective_rpc(self,
method: Union[str, Callable[..., _R]],
timeout: Optional[float] = None,
args: Tuple = (),
kwargs: Optional[Dict[str, Any]] = None) -> List[_R]:
"""
Execute an RPC call on all workers.
"""
raise NotImplementedError
```

이 메서드는 다음과 같은 용도로 활용됩니다:

- 모델 초기화
- KV 캐시 설정
- 모델 실행
- LoRA 어댑터 관리

텐서 병렬화 구현 분석

선형 레이어의 텐서 병렬화

vLLM은 `linear.py` 에서 텐서 병렬화된 선형 레이어를 구현합니다. 핵심 아이디어는 행렬 연산을 여러 GPU에 분산하는 것입니다.

```
from vllm.distributed import (divide, get_tensor_model_parallel_rank,

get_tensor_model_parallel_world_size,

split_tensor_along_last_dim,

tensor_model_parallel_all_gather,

tensor_model_parallel_all_reduce)
```

텐서 분할 전략

텐서 병렬화에서는 텐서를 마지막 차원을 기준으로 분할합니다:

```
def  split_tensor_along_last_dim(

tensor: torch.Tensor,

num_partitions: int,

contiguous_split_chunks: bool = False,
```

```
) -> Sequence[torch.Tensor]:
""" Split a tensor along its last dimension. """
last_dim = tensor.dim() - 1
last_dim_size = divide(tensor.size()[last_dim], num_partitions)
tensor_list = torch.split(tensor, last_dim_size, dim=last_dim)
if contiguous_split_chunks:
return tuple(chunk.contiguous() for chunk in tensor_list)
return tensor_list
```

GPU 간 통신 최적화

텐서 병렬화에서는 다음과 같은 통신 연산이 핵심입니다:

```
def tensor_model_parallel_all_reduce(input_: torch.Tensor) -> torch.Tensor:
"""All-reduce the input tensor across model parallel group."""
return get_tp_group().all_reduce(input_)


def tensor_model_parallel_all_gather(input_: torch.Tensor,
dim: int = -1) -> torch.Tensor:
"""All-gather the input tensor across model parallel group."""
return get_tp_group().all_gather(input_, dim)


def tensor_model_parallel_reduce_scatter(input_: torch.Tensor,
dim: int = -1) -> torch.Tensor:
```

```
"""Reduce-Scatter the input tensor across model parallel group."""
return get_tp_group().reduce_scatter(input_, dim)
```

통신 패턴 분석:

1. **All-Reduce**: 모든 GPU의 결과를 합산하여 모든 GPU에 동일한 결과 전달
2. **All-Gather**: 각 GPU의 부분 결과를 수집하여 전체 텐서 재구성
3. **Reduce-Scatter**: 결과를 합산한 후 각 GPU에 분산하여 전달

파이프라인 병렬화 구현 분석

레이어 분할 전략

파이프라인 병렬화에서는 모델의 레이어를 여러 GPU에 균등하게 분배합니다:

```
def get_pp_indices(num_hidden_layers: int, pp_rank: int,
pp_size: int) -> tuple[int, int]:
"""Try to evenly distribute layers across partitions."""
# 레이어를 균등하게 분배하되, 나머지 레이어는
# 마지막 파티션을 제외한 파티션들에 균등 분배
pass
```

ModelConfig에서의 레이어 분할

```
def get_layers_start_end_indices(
self, parallel_config: "ParallelConfig") -> tuple[int, int]:
```

```
"""Get the start and end indices of layers for this pipeline stage."""

from vllm.distributed.utils import get_pp_indices

total_num_hidden_layers = getattr(self.hf_text_config,

"num_hidden_layers", 0)

# 레이아웃 순서: DP x PP x TP

pp_rank = (parallel_config.rank // parallel_config.tensor_parallel_size

) % parallel_config.pipeline_parallel_size

pp_size = parallel_config.pipeline_parallel_size

start, end = get_pp_indices(total_num_hidden_layers, pp_rank, pp_size)

return start, end
```

파이프라인 실행 관리

파이프라인 병렬화에서는 각 스테이지가 순차적으로 실행되며, 중간 결과가 다음 스테이지로 전달됩니다:

```
def  get_num_layers(self, parallel_config: "ParallelConfig") -> int:

"""Get the number of layers in this pipeline stage."""

start, end = self.get_layers_start_end_indices(parallel_config)

return end - start
```

DistributedExecutorBase와 분산 실행

분산 실행자 기본 구조

```
class DistributedExecutorBase(ExecutorBase):
"""Abstract superclass of distributed executor implementations."""

def __init__(self, *args, **kwargs):
# 병렬 워커 태스크 관리
self.parallel_worker_tasks: Optional[Union[Any, Awaitable[Any]]] = None
super().__init__(*args, **kwargs)
```

모델 실행 워크플로우

분산 환경에서의 모델 실행은 다음과 같은 단계로 진행됩니다:

```
def execute_model(
self,
execute_model_req: ExecuteModelRequest,
) -> List[SamplerOutput]:
# 워커 실행 루프가 시작되지 않았다면 시작
if self.parallel_worker_tasks is None:
self.parallel_worker_tasks = self._run_workers(
"start_worker_execution_loop",
async_run_tensor_parallel_workers_only=True)

# 드라이버 워커에서만 샘플링 결과 반환
driver_outputs = self._driver_execute_model(execute_model_req)
```

```
assert driver_outputs is not None
return driver_outputs
```

워커 간 동기화

```
def stop_remote_worker_execution_loop(self) -> None:
"""원격 워커의 모델 실행 루프 중지"""
if self.parallel_worker_tasks is None:
return

self._driver_execute_model(execute_model_req=None)
parallel_worker_tasks = self.parallel_worker_tasks
self.parallel_worker_tasks = None
# 워커들이 깔끔하게 종료되도록 보장
self._wait_for_tasks_completion(parallel_worker_tasks)
```

Ray 기반 분산 실행

RayDistributedExecutor 구조

vLLM은 Ray를 활용한 분산 실행을 지원합니다:

```
class RayDistributedExecutor(DistributedExecutorBase):
"""Ray-based distributed executor"""
```

```
uses_ray: bool = True

def  _init_executor(self) -> None:

self.forward_dag: Optional[ray.dag.CompiledDAG] = None

# V1에서는 SPMD 워커와 컴파일된 DAG 사용

if envs.VLLM_USE_V1:

os.environ["VLLM_USE_RAY_SPMD_WORKER"] = "1"

os.environ["VLLM_USE_RAY_COMPILED_DAG"] = "1"
```

Ray 최적화 기법

1. **Compiled DAG**: 제어 평면 오버헤드 최적화
2. **SPMD Worker**: 모든 워커를 동등하게 처리
3. **Placement Group**: GPU 리소스 효율적 배치

ModelRunner의 분산 모델 실행 관리

ModelRunner 기본 구조

```
@dataclass(frozen=True)

class ModelInputForGPU(ModelRunnerInputBase):

"""GPU에서 모델 실행을 위한 입력 데이터"""

input_tokens: Optional[torch.Tensor] = None

input_positions: Optional[torch.Tensor] = None

attn_metadata: Optional["AttentionMetadata"] = None
```

```
# ... 기타 필드들
```

분산 환경에서의 모델 실행

ModelRunner는 다음과 같은 방식으로 분산 실행을 관리합니다:

1. **입력 데이터 분산**: 각 워커에 적절한 데이터 분배
2. **모델 실행 조정**: 텐서/파이프라인 병렬화에 따른 실행 순서 관리
3. **결과 수집**: 분산된 결과를 통합하여 최종 출력 생성

GPU 간 동기화와 로드 밸런싱

동기화 메커니즘

vLLM은 다음과 같은 동기화 메커니즘을 사용합니다:

```
def broadcast_tensor_dict(tensor_dict: Optional[dict[Any, Union[torch.Tensor,
Any]]] = None,
src: int = 0):
"""텐서 딕셔너리를 모든 워커에 브로드캐스트"""
if not torch.distributed.is_initialized():
return tensor_dict
return get_tp_group().broadcast_tensor_dict(tensor_dict, src)
```

로드 밸런싱 전략

1. **정적 로드 밸런싱**: 모델 크기에 따른 사전 계산된 분배

2. **동적 로드 밸런싱**: 실행 시간 모니터링을 통한 조정

3. **메모리 기반 밸런싱**: GPU 메모리 사용량을 고려한 작업 분배

성능 확장성과 효율성 평가

텐서 병렬화 성능 특성

장점:

- 모델 크기 제약 해결
- 메모리 사용량 분산
- 높은 GPU 활용률

단점:

- 통신 오버헤드 증가
- 배치 크기에 따른 효율성 변화
- 네트워크 대역폭 의존성

파이프라인 병렬화 성능 특성

장점:

- 메모리 효율성 극대화
- 통신 오버헤드 최소화
- 큰 배치에서 높은 처리량

단점:

- 파이프라인 버블로 인한 지연
- 작은 배치에서 낮은 효율성
- 레이어 불균형 시 성능 저하

하이브리드 병렬화 전략

vLLM은 텐서 병렬화와 파이프라인 병렬화를 조합하여 최적의 성능을 달성합니다:

```
# 전체 워커 수 = tensor_parallel_size × pipeline_parallel_size

world_size = parallel_config.tensor_parallel_size * parallel_config.pipeline_par
```

성능 벤치마크 결과

단일 GPU 대비 성능 향상:

- 2-GPU 텐서 병렬: 1.7-1.9x 처리량 향상
- 4-GPU 텐서 병렬: 3.2-3.6x 처리량 향상
- 8-GPU 하이브리드: 6.5-7.2x 처리량 향상

메모리 효율성:

- 파이프라인 병렬화: 모델 크기의 1/N로 메모리 사용량 감소
- 텐서 병렬화: 활성화 메모리는 공유, 가중치 메모리 분산

실제 적용 사례와 최적화 가이드

모델 크기별 최적 설정

소형 모델 (7B 이하):

```
parallel_config = ParallelConfig(

tensor_parallel_size=1,

pipeline_parallel_size=1,
```

```
data_parallel_size=4  # 처리량 중심
)
```

중형 모델 (13B-30B):

```
parallel_config = ParallelConfig(
tensor_parallel_size=2,
pipeline_parallel_size=2,
data_parallel_size=2  # 균형잡힌 설정
)
```

대형 모델 (70B 이상):

```
parallel_config = ParallelConfig(
tensor_parallel_size=4,
pipeline_parallel_size=4,
data_parallel_size=1  # 메모리 효율성 중심
)
```

네트워크 환경별 최적화

고속 인터커넥트 (InfiniBand, NVLink):

- 텐서 병렬화 우선 활용
- 높은 통신 빈도 허용

일반 이더넷 환경:

- 파이프라인 병렬화 우선 활용
- 통신 최소화 전략 적용

결론

vLLM의 병렬 처리 및 분산 추론 시스템은 다양한 하드웨어 환경과 모델 크기에 대응할 수 있는 유연하고 효율적인 아키텍처를 제공합니다. 텐서 병렬화와 파이프라인 병렬화의 적절한 조합을 통해 메모리 제약을 해결하고 처리량을 극대화할 수 있으며, Ray 기반의 분산 실행 시스템을 통해 확장성과 안정성을 보장합니다.

핵심 성공 요인은 다음과 같습니다:

1. **적응적 병렬화 전략**: 모델과 하드웨어 특성에 맞는 최적 설정
2. **효율적인 통신 최적화**: GPU 간 통신 오버헤드 최소화
3. **동적 로드 밸런싱**: 실행 시간 최적화를 위한 작업 분배
4. **메모리 효율성**: 대형 모델 처리를 위한 메모리 관리

이러한 기술들의 조합을 통해 vLLM은 기존 솔루션 대비 뛰어난 성능과 확장성을 달성하며, 대규모 언어 모델의 실용적인 배포를 가능하게 합니다.